

How to Approximate A Set Without Knowing Its Size In Advance

241880161 仇浩嘉 241880136 任春驿 241880070 周文成

2026 年 7 月 27 日

目录

1	论文解决了什么问题	2
2	这个问题为什么重要	2
3	相关研究	2
4	主要定理与贡献	4
5	下界证明思路	5
6	上界构造细节	9
7	总结与展望	12

1 论文解决了什么问题

1.1 经典问题

Approximate membership data structures: 给定一个可以在线插入元素的集合 S (大小为 n), 支持成员查询, 并且需要满足以下这两个条件:

- 无假阴性: $\forall x \in S$, 查询总是返回 yes。
- 假阳性率最多 ε : $\forall x \notin S$, 查询返回 yes 的概率最多达到给定的 ε 。

当 n 已知的时候, 理论空间占用是 $\Theta(n \log(1/\varepsilon))$ bits, 单个元素占用 $\Theta(\log(1/\varepsilon))$ bits。

1.2 本文研究的变体

已有的 Approximate membership data structures 需要集合 S 的大小 n 已知, 该论文研究的是 n 未知的情形。最终集合大小 n 不定, 需要考虑结构自适应扩容, 且要维持空间尽可能小。传统最优空间理论在这个设定下不再直接适用。

2 这个问题为什么重要

现代计算环境本质上是流式计算:

- 数据随机: 无法预知会有多少数据量。
- 突发流量: 即使有历史统计平均值作参考, 也难免有时出现数据量激增导致 n 的值翻倍。
- 如果数据结构严格要求预先知道 n 的大小, 在流量激增的时候要么拒绝服务, 要么假阳性率失控。

在大数据流场景(网络监控、数据库缓存、分布式系统)中, 集合大小往往无法提前预知。这篇论文解释了为什么简单扩展 Bloom Filter 会浪费空间, 并为”空间自适应”的近似成员结构提供了理论指导。

3 相关研究

3.1 Bloom filters (布隆过滤器)

- 布隆过滤器是一种很经典的数据结构, 天然支持动态插入。其空间开销是信息论下界 $n \log(1/\varepsilon)$ 的 $\log(e) \approx 1.44$ 倍。
- 标准的 Bloom 过滤器不支持从集合 S 中删除元素, 因为将任意位重置为 0 可能会导致假阴性。

- 为了支持删除查询，可以使用计数布隆过滤器。但这会使空间使用增加 $\Omega(\log \log n)$ 倍。这种删除支持是有条件的：只有在不删除假阳性元素时，数据结构才能正确工作，但根据定义，无法完全避免对假阳性元素的删除。
- Cohen 和 Matias 提出了一种方法，将空间开销降低至 $O(n)$ 位。

3.2 基于字典的近似成员查询

- 早在 1978 年，Carter 等人就提出了一种能达到类似结果的技术。他们观察到，维护一个多重集 $h(S)$ （其中 $h: [u] \rightarrow [n/\varepsilon]$ 是一个通用哈希函数），如果集合 $h(S)$ 的存储空间接近信息论下界 $\log \binom{n/\varepsilon+n}{n}$ 位，那么整体解决方案的空间消耗仅为 $n \log(1/\varepsilon) + O(n)$ 位。
- 如果不需要支持删除操作，只需存储不同的哈希值集合 $h(S)$ 即可。这种动态集合可以被简洁地存储，且所有操作以高概率在 $O(1)$ 时间内完成。如果需要支持动态多重集（即支持删除），可以通过转化为标准成员查询问题来实现，代价是更新操作的时间增大。

3.3 在线/离线空间需求的差异

- 在静态情况下，已经证明能够逼近 $n \log(1/\varepsilon)$ 的空间下界。例如，Dietzfelbinger 和 Pagh 证明了在查询时间为 $\omega(\log(1/\varepsilon))$ 且 ε 为 2 的整数幂时，空间开销可以控制在 $o(n)$ 项以内；Porat 在常数查询时间的情况下实现了相同的结果；随后 Bellazougui 和 Venturini 进一步消除了对 ε 的限制，同时维持了常数查询时间。
- 然而，Lovett 和 Porat 证明了这些静态情况下的结果无法直接扩展到允许动态场景：在动态更新下，需要 $\Omega(n/\log(1/\varepsilon))$ 位的额外空间开销。即使在整个插入序列结束前不进行任何查询，该下界依然成立。这意味着，对于以数据流形式给出的键集，仅使用接近静态大小下界的空间来构建近似成员数据结构是不够的。

3.4 动态空间使用

- 当要求空间必须依赖于集合的当前大小时，上界要求更加严格。
- 文献中现有的相关技术通常会导致 $\Omega(\log n)$ 或 $\Omega(\log(1/\varepsilon))$ 的查询时间，以及 $\Omega(n \log n)$ 位的高空间开销。
- 这些数据结构通常采用维护多个近似成员数据结构并逐一查询的策略。如果采用几何级数递增的容量，就需要 $\Omega(\log n)$ 个这样的数据结构。每一个结构对应的假阳性率 $\varepsilon_1, \varepsilon_2, \dots$ 之和必须收敛于目标 ε 。例如，在已有的 Scalable Bloom Filter 中让 ε_i 随 i 呈几何级数递减，会导致 $\varepsilon_i = n^{-\Omega(i)}$ ，从而产生 $\Omega(n \log n)$ 的空间使用量——渐近上远非最优。

3.5 动态完美哈希与检索

- 在静态情况下，一种实现近似成员查询的方法是存储一个完美哈希函数，将键单射地映射到 $\{1, \dots, n\}$ ，然后在数组中根据该完美哈希函数为每个键存储 $\log(1/\varepsilon)$ 位的签名。

- 更一般地，动态检索数据结构（如 Bloomier filters）也可以用来构建动态近似成员数据结构。研究表明，在线设置下，这两个问题都需要 $\Theta(n \log \log n)$ 的空间。然而，这些上界具有固定的空间使用量（在常数因子内），因此无法实现本文所获得的结果。

4 主要定理与贡献

论文在集合大小 n 事先未知的设定下，给出了近似成员查询问题的空间下界和上界，并给出两种有效构造。

4.1 定理 1.1（下界）

任意针对未知大小集合 S 、假阳性率 ε 的近似成员数据结构，在某 $n > u^\delta$ 次插入后，必须使用至少

$$(1 - o(1))n \log(1/\varepsilon) + \Omega(n \log \log n)$$

位的空间。

- 第一项 $(1 - o(1))n \log(1/\varepsilon)$ 是已知 n 时就已经需要的。
- 第二项 $\Omega(n \log \log n)$ 是”不知道 n ”所付出的额外代价。

注：条件 $n > u^\delta$ 的意思是 $\delta \in (0, 1)$ 为任意小常数， n 与全集大小 u 之间呈多项式关系（如 $n > \sqrt{u}$ 或 $n > u^{0.01}$ ），排除 n 极小（如 $n = O(\log u)$ ）的退化情况。需要注意，相比传统已知规模的场景，这里每元素平均开销多出了 $\Omega(\log \log n)$ bits，意味着开销会随着集合规模增长，无法维持常数单元素存储。在实际场景中假阳性率常取一个不太小的常数（如 $\varepsilon = 1/10$ ），此时下界要求每元素 $\Omega(\log \log n)$ 比特，而不是传统设定中的常数。

4.2 定理 1.2（上界）

论文给出两种匹配下界的构造，以及构造 2 的去摊销优化版本：

属性	构造 1（热身）	构造 2（精妙）	构造 2（去摊销）
空间	$(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$	$(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$	$O(n \log(1/\varepsilon) + n \log \log n)$
假阳性率	$\leq \varepsilon$	$\leq \varepsilon + u^{-c}$	$\leq \varepsilon + u^{-c}$
假阴性	无	无	无
插入时间	期望摊还 $O(1)$	期望摊还 $O(1)$	最坏 $O(1)$
查询时间	$O(\log n)$	最坏 $O(1)$	最坏 $O(1)$
是否需要已知 n	否	否	否

4.3 与 Bloom Filter 的对比

	Bloom Filter	构造 1	构造 2	构造 2 去摊销
空间（每元素）	$1.44 \log(1/\varepsilon)$	$(1 + o(1)) \log(1/\varepsilon) + O(\log \log n)$	同构造 1	$O(\log(1/\varepsilon) + \log \log n)$
已知 n ?	是	否	否	否
查询时间	$O(\log(1/\varepsilon))$	$O(\log n)$	最坏 $O(1)$	最坏 $O(1)$
插入时间	$O(\log(1/\varepsilon))$	期望摊还 $O(1)$	期望摊还 $O(1)$	最坏 $O(1)$
空间是否紧致	否 ($1.44\times$)	是	是	否（常数损失）
结构数量	1	$\log n$	1	1
桶化	不适用	不需要	用 ($u^{\delta/2}$ 桶)	不能用
依赖的哈希	k 个独立哈希	通用哈希	成对独立	成对独立
支持删除	否	否	可（扩展）	可（扩展）

下界为 $\Omega(n \log(1/\varepsilon) + n \log \log n)$ ，上界为 $O(n \log(1/\varepsilon) + n \log \log n)$ ，两者渐近阶完全一致。说明未知规模带来的 $\log \log n$ 的平均开销是不可避免的，也因此不存在更低阶的数据结构。

5 下界证明思路

下界证明用的是压缩论证：把数据结构当成编码工具，如果空间太小，就能用数据结构的把插入序列压缩到信息论下界以下，推出矛盾。证明的核心链路是：随机结构转确定性 → 指数分段 → 找测度增长最慢的一段 → 控制重叠元素 → 构造四段无损编码 → 化简得下界 → 参数赋值得主定理。

5.1 3.1：随机结构转化为确定性结构

我们定义几个东西：

- 随机比特串 r ：数据结构内部所有哈希、随机选择依赖的只读随机源，存储空间不计入结构开销。
- 确定性结构 B_r ：将随机源 r 固定后，整个插入、查询流程不再存在任何随机行为，输入完全决定输出。
- 判定超集 $\hat{S}_r$ ：对于插入序列 S ，所有被 B_r 查询返回“存在”的元素构成的集合。无假阴性保证 $S \subseteq \hat{S}_r$ 。
- 归一化测度 $\mu(\hat{S}_r) = |\hat{S}_r|/u$ ：判定超集在全集 U 中的占比，等价于随机抽一个元素被判 Yes 的概率。
- 合法序列集合 $S_{r,\varepsilon} = \{S \in U^n \mid \mu(\hat{S}_r) \leq 4\varepsilon\}$ ：固定随机种子 r 后，假阳性概率不超过 4ε 的那些插入序列。

引理 3.2: 存在随机串 r^* , 使 $|S_{r^*, \varepsilon}| \geq u^n/2$ 。

证明:

1. 对任意长度 n 的插入序列 S , 原始随机结构要求假阳性概率 $\leq \varepsilon$, 所以:

$$\mathbb{E}_{r \leftarrow \{0,1\}^*} [\mu(\hat{S}_r)] \leq \varepsilon + \frac{n}{u}$$

文中限定 $n \leq \varepsilon u$, 所以 $n/u \leq \varepsilon$, 右式 $\leq 2\varepsilon$ 。

2. 用马尔可夫不等式: $\mu(\hat{S}_r) \geq 0$ 为非负随机变量, 取 $a = 4\varepsilon$:

$$\Pr [\mu(\hat{S}_r) \geq 4\varepsilon] \leq \frac{\mathbb{E}[\mu(\hat{S}_r)]}{4\varepsilon} \leq \frac{2\varepsilon}{4\varepsilon} = \frac{1}{2}$$

3. 全空间 U^n 有 u^n 条插入序列。对任意 r , 落入 $S_{r, \varepsilon}$ 的序列比例至少 $1/2$ 。所以存在某个 r^* 使得至少一半的序列满足 $\mu(\hat{S}_{r^*}) \leq 4\varepsilon$ 。

5.2 3.2: 压缩论证

拿到确定性结构 B_{r^*} 与合法序列集合 $\mathcal{S} = S_{r^*, \varepsilon}$ 后, 开始压缩论证。对任意长度为 n 的序列 S , 做指数分段:

- 分段基数 $\gamma \geq 2$ (自定义参数, 控制分段长度增长速度)。
- 第 i 段子序列 C_i 长度严格为 γ^i 。
- 前缀 S_i : 前 i 段拼接, 长度 $n_i = \sum_{j=1}^i \gamma^j$ 。
- 超集单调性: $\hat{S}_1 \subseteq \hat{S}_2 \subseteq \dots$, 每次插入新元素只能扩大判定超集, 所以测度单调不减: $0 \leq \mu(\hat{S}_1) \leq \dots \leq 4\varepsilon$ 。

分段的目的: 把完整的插入流程拆成指数增长的片段, 从中截取一段 C_i 作为编码论证的分析单元, 用总测度增量平摊到各段来得到单段增量的上界。

5.2.1 引理 3.3: 找测度增长最慢的一段

命题: 对任意 $S \in \mathcal{S}$, 存在 i 满足前缀长度 $n_i \in [\alpha n, n]$, 且:

$$\mu(\hat{S}_i) - \mu(\hat{S}_{i-1}) \leq \frac{4\varepsilon}{\log_\gamma(1/\alpha) - 2}$$

证明就是抽屉原理: 划定区间上下界 j_1, j_2 保证所有 $i \in [j_1, j_2]$ 对应的 n_i 落在 $(\alpha n, n)$ 内。总测度增量上限 4ε , 区间共 $j_2 - j_1 \geq \log_\gamma(1/\alpha) - 2$ 个分段增量, 必有一段增量不超过平均值。

5.2.2 引理 3.4：控制重叠元素

定义 $k = |C_i \cap \hat{S}_{i-1}|$, C_i 中在插入前就已被判 Yes 的元素数。

命题： 满足 $k(S) \leq 9\varepsilon|C_i|$ 的序列至少占 $u^n/3$ 。

证明：

1. 假设序列逐段均匀独立从全集 U 采样。 C_i 均匀随机, $\mathbb{E}[k] = |C_i| \cdot \mu(\hat{S}_{i-1}) \leq 4\varepsilon|C_i|$ 。
2. 用切尔诺夫界：取 $(1 + \delta) \cdot 4\varepsilon = 9\varepsilon$, 得 δ 为常数, 尾概率 $\Pr[k \geq 9\varepsilon|C_i|] \leq \exp(-|C_i|)$ 。
3. $|C_i| = n^{\Omega(1)}$, 单段超标的概率指数级小。
4. 对所有分段用联合界, 结合引理 3.2 的 $|S| \geq u^n/2$, 最终满足 $k \leq 9\varepsilon|C_i|$ 的序列至少 $u^n/3$ 。

5.2.3 四段式无损编码

对任意合法序列 S , 设计一套无损编码, 解码器仅靠编码串就能完整还原 S :

1. 下标 i : $\log \log n$ 比特。
2. C_i 以外的全部元素：原始完整存储, $(n - c_i) \log u$ 比特 ($c_i = |C_i|$)。
3. 完整数据结构内部状态: b_i 比特 (含公共随机源 r^*)。解码器拿到前两段加上 b_i 位状态码, 可以重建 B_{r^*} 在处理 C_i 前后的内部状态, 还原 $\hat{S}_{i-1}$ 和 $\hat{S}_i$ 两个超集。
4. C_i 的相对压缩编码, 分两类:
 - k 个重叠元素 (落在旧超集 $\hat{S}_{i-1}$ 内): 在 $\mu(\hat{S}_i) \cdot u$ 范围内定位, 开销 $k \cdot \log(\mu(\hat{S}_i) \cdot u) + O(1)$ 。
 - $c_i - k$ 个新增元素 (落在 $\hat{S}_i \setminus \hat{S}_{i-1}$ 内): 在新增的超集区间内定位, 候选空间更小, 开销 $(c_i - k) \cdot \log(\Delta\mu \cdot u) + O(1)$ 。

总编码长度:

$$\text{TotalLen} = \log \log n + (n - c_i) \log u + b_i + (c_i - k) \log(\Delta\mu \cdot u) + k \log(\mu(\hat{S}_i)u) + O(c_i)$$

5.2.4 化简与矛盾

把 $\log(\mu u)$ 拆成 $\log u + \log \mu$, 提取公共 $\log u$ 项:

$$\text{TotalLen} = b_i + \log \log n + n \log u + (c_i - k) \log \Delta\mu + k \log \mu(\hat{S}_i) + O(c_i)$$

代入三个引理的约束:

- 引理 3.3: $\log \Delta\mu \leq \log \varepsilon - \log \log_\gamma(1/\alpha) + O(1)$
- $\mu(\hat{S}_i) \leq 4\varepsilon \leq \varepsilon$, 所以 $\log \mu(\hat{S}_i) \leq \log \varepsilon$

- 引理 3.4: $k \leq 9\varepsilon c_i$

合并包含 c_i 的项:

$$(c_i - k) \log \Delta \mu + k \log \mu(\hat{S}_i) \leq c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1))$$

得到简化后的上界:

$$\text{TotalLen} \leq b_i + \log \log n + n \log u + c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1))$$

无损编码要求能区分 $|\mathcal{S}| \geq u^n/3$ 条不同序列, 编码长度必须 $\geq \log(u^n/3) = n \log u - O(1)$ 。
令上界 $\geq$ 下界:

$$b_i + \log \log n + n \log u + c_i (\log \varepsilon - (1 - 9\varepsilon) \log_\gamma(1/\alpha) + O(1)) \geq n \log u - O(1)$$

消去 $n \log u$, 移项, $\log \log n$ 吸收进 c_i 的常数项, 得单段空间下界:

$$b_i \geq c_i \left(\log \frac{1}{\varepsilon} + (1 - 9\varepsilon) \log_\gamma \frac{1}{\alpha} - O(1) \right)$$

5.2.5 完成定理 3.1

$c_i = \gamma^i$, 等比数列求和:

$$n_i = \gamma \frac{\gamma^i - 1}{\gamma - 1} = c_i \cdot \frac{\gamma - 1/\gamma^{i-1}}{\gamma - 1}$$

$i \rightarrow \infty$ 时 $1/\gamma^{i-1} \rightarrow 0$, 得 $c_i \approx n_i \cdot (\gamma - 1)/\gamma$, 即 $c_i = \Theta(n_i)$ 。

代入反证假设 $b_i \leq \beta n_i$, 约去 n_i :

$$\beta \geq \left(1 - \frac{1}{\gamma}\right) \cdot \left(\log \frac{1}{\varepsilon} + (1 - 9\varepsilon) \log_\gamma \frac{1}{\alpha} - \Theta(1)\right)$$

5.2.6 导出主定理 1.1

- $\alpha = 1$ 时退化为已知规模: 令 $\gamma \rightarrow \infty$, $\beta \geq (1 - o(1)) \log(1/\varepsilon)$ ——和传统下界一致, 自洽性没问题。
- $\alpha = 1/\sqrt{n}$, $\gamma = 2^{(\log n)^\eta}$ ($\eta \in (0, 1)$):

$$\log_\gamma \frac{1}{\alpha} = \log_{2^{(\log n)^\eta}} \sqrt{n} = \frac{\frac{1}{2} \log n}{(\log n)^\eta} = \frac{1}{2} (\log n)^{1-\eta} = \Theta(\log \log n)$$

回代 $S = \beta n$:

$$S \geq (1 - o(1)) n \log \frac{1}{\varepsilon} + \Omega(n \log \log n)$$

5.2.7 证明链路小结

1. 原始随机近似结构 $\rightarrow$ 马尔可夫不等式 (引理 3.2) $\rightarrow$ 固定 r^* 转为确定性结构
2. 对插入序列指数分段 $C_i, S_i \rightarrow$ 抽屉原理 (引理 3.3): 单段超集增量可控
3. 均匀采样 + 切尔诺夫界 (引理 3.4): 绝大多数分段重叠元素数量可控
4. 构造四段无损编码 $\rightarrow$ 信息论编码下界建立矛盾不等式
5. 化简得到单段 b_i 下界 $\rightarrow$ 结合 c_i, n_i 等比关系证明定理 3.1
6. 代入两组参数 $\rightarrow$ 分别得到已知/未知规模空间下界 (定理 1.1)

6 上界构造细节

6.1 预备知识

6.1.1 单位代价 Word RAM 模型

全集大小为 u , 单个元素占一个字, 字长 $w = \lceil \log u \rceil$ bits。元素加减、按位布尔运算、任意位数的移位、整数乘法都是单位耗时。这是数据结构上下界证明的通用标准模型。下界证明不关心操作时间, 只在意存储空间比特数。

6.1.2 k 重独立哈希函数族

$H : U \rightarrow V$ 是 k -wise independent 的, 如果任取 k 个互不相同的 $x_1, \dots, x_k$ 和任意 $y_1, \dots, y_k \in V$:

$$\Pr_{h \leftarrow H}[h(x_1) = y_1 \wedge \dots \wedge h(x_k) = y_k] = \frac{1}{|V|^k}$$

弱化版 k -wise δ -dependent: $(h(x_1), \dots, h(x_k))$ 与均匀分布的统计距离不超过 δ , 允许微小偏差, 降低实现开销。

论文构造 2 只需要 pairwise independent ($k = 2$), 计算常数时间, 仅需 $O(\log u)$ 位随机种子。

6.1.3 四个概率工具

- **马尔可夫不等式**: 非负 $X \geq 0$, $\Pr[X \geq a] \leq \mathbb{E}[X]/a$ 。引理 3.2 中用来筛选优质随机种子: $\mathbb{E}[\mu(\hat{S}_r)] \leq 2\varepsilon$, 取 $a = 4\varepsilon$, 得 $\Pr[\mu \geq 4\varepsilon] \leq 1/2$ 。
- **切尔诺夫界**: 独立伯努利变量和偏离期望的概率指数衰减, 精度远高于马尔可夫。引理 3.4 中, $k = |C_i \cap \hat{S}_{i-1}|$ 期望 $\leq 4\varepsilon|C_i|$, 取 $(1 + \delta)\mu = 9\varepsilon|C_i|$ 得尾概率 $\exp(-|C_i|)$, $|C_i| = n^{\Omega(1)}$ 时趋近于 0。

- **联合界**: $\Pr[\bigcup_i A_i] \leq \sum_i \Pr[A_i]$ 。构造 1 中用于多子过滤器总假阳性控制 ($\sum \varepsilon_i \leq \varepsilon$)，引理 3.4 中证明所有分段重叠超标的总概率极小。
- **无损编码信息论下界**: M 条互不相同的序列用无损前缀编码，至少需要 $\log_2 M$ bits。合法序列集合规模 $\geq u^n/3$ ，所以编码总长度必须 $\geq \log(u^n/3)$ 。压缩论证的核心武器。

6.2 构造 1：几何增长数据结构（热身）

6.2.1 思路

把插入流按指数分块——第 i 块 2^i 个元素，每块由专门的 B_i 处理。共需要约 $\log n$ 个 B_i 。第 i 个子序列开始时分配并初始化 B_i ，假阳性率 $\varepsilon_i = \Theta(\varepsilon/i^2)$ 。查询时逐个问所有 B_i ，任一返回 Yes 就判定存在。

6.2.2 B_i 内部设计

每个 B_i 不是全新发明，而是套用已知的 dictionary-based approximate membership 方法，底层用 Raman and Rao [RR03] 的动态字典：

- 选哈希函数 $h: U \rightarrow [m]$ ， $m \approx 2^i/\varepsilon_i$ 。
- 插入 x ：算 $h(x)$ ，把 $h(x)$ 存入 Raman-Rao 字典。
- 查询 x ：算 $h(x)$ ，查字典里有没有。
- Raman-Rao 字典：期望摊还 $O(1)$ 插入，最坏 $O(1)$ 查询，空间 $(1 + o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i)$ bits。

6.2.3 插入和查询

插入：将要插入第 m 个元素时，通过 $2^i - 2 < m \leq 2^{i+1} - 2$ 确定该去哪个 B_i 。如果是该块第一个元素就新建 B_i ，否则直接插入。绝大多数插入是直接插入，新建 B_i 的时间被摊销——期望摊还 $O(1)$ 。

查询：对当前所有已存在的 $B_1, \dots, B_k$ 逐个查询，任一 Yes 则返回 Yes，全 No 才 No。 $k \approx \log n$ ，总查询 $O(\log n)$ 。

假阳性：由联合界 $\leq \sum_{i=1}^{\infty} \varepsilon_i = \Theta(\varepsilon \cdot \pi^2/6) \approx \varepsilon$ 。

6.2.4 空间复杂度

单个 B_i 空间 $(1 + o(1)) \cdot 2^i \cdot \log(1/\varepsilon_i)$ 。代入 $\varepsilon_i = \Theta(\varepsilon/i^2)$ ：

$$\log(1/\varepsilon_i) = \log(1/\varepsilon) + 2\log i + O(1)$$

$i \leq \log n$, $\max_i \{\log(1/\varepsilon_i)\} = \log(1/\varepsilon) + O(\log \log n)$ 。总空间:

$$(1 + o(1)) \cdot n \cdot \max_i \{\log(1/\varepsilon_i)\} = (1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$$

匹配下界。

优点: 简单直观。**缺点:** 查询 $O(\log n)$, 因为不知道元素属于哪个子序列。

6.3 构造 2: 常数时间操作

6.3.1 核心思路

和构造 1 完全不同的路线——任何时候只维护一个动态字典 D_i (精确集合), 存储当前已插入元素的某个超集。使用 pairwise independent 哈希 $h: U \rightarrow \{0, 1\}^\ell$, $\ell \geq \lceil \log(1/\varepsilon) \rceil + \log u + 2$ 。

同样将插入序列按 2^i 分块。对每个阶段 i 的元素 x , 在 D_i 中存储 $(h_i(x), g_i(x))$:

- $h_i(x)$: $h(x)$ 的最左 $\ell_i = \lceil \log(1/\varepsilon) \rceil + i + 2$ 位, 作为字典键。
- $g_i(x)$: $h(x)$ 的接下来 $r = \lceil \log \log u \rceil$ 位, 不足时以 $\perp$ 填充。

6.3.2 插入: 两种模式

Mode 1 (普通插入): 当前元素不是子序列的第一个。直接把 $(h_i(x), g_i(x))$ 以 $h_i(x)$ 为键插入当前 D_i 。 $O(1)$ 。

Mode 2 (阶段过渡): 当前元素是第 i 个子序列的第一个。做字典切换:

1. 初始化 D_i : 容量 2^{i+2} 项, 每项 $\ell_i + r$ 位。
2. 枚举 D_{i-1} 中所有记录 $(y, \alpha_1 \cdots \alpha_r)$ 迁移:
 - 若 $\alpha_1 \neq \perp$: 插入 $(y\alpha_1, \alpha_2 \cdots \alpha_r \perp)$, 扩展一位签名。
 - 若 $\alpha_1 = \perp$: 分支, 插入 $(y0, \alpha)$ 和 $(y1, \alpha)$ 。
3. 释放 D_{i-1} , 然后按 Mode 1 插入当前元素。

6.3.3 正确性

不溢出: 早期子序列 $(S_1, \dots, S_{i-r})$ 标签耗尽后每个 S_j 元素贡献 2^{i-j-r} 条记录; 近期子序列 $(S_{i-r+1}, \dots, S_i)$ 标签尚有剩余, 每元素 1 条。总条目 $\leq 2^{i+2}$, 字典不溢出。

无假阴性: 若 $\alpha_1 = \perp$ 则同时保留 $y0$ 和 $y1$, 覆盖了 $h_i(x)$ 所有可能前缀。 x 在任意后续阶段都至少有 $(h_i(x), \cdot)$ 在 D_i 中, 查询一定命中。

假阳性 $\leq \varepsilon + u^{-c}$: D_i 中最多 2^{i+2} 个键, pairwise independent 保证 $h_i(x)$ 和已有键碰撞的概率 $\leq 2^{i+2} \cdot 2^{-\ell_i} \leq \varepsilon$ 。底层字典失败概率通过重建或把前 u^δ 个元素合并处理, 控制在 u^{-c} 以内。

6.3.4 查询

当前阶段 i ，用 $h_i(x)$ 查 D_i ，命中则 Yes，否则 No。查询最坏 $O(1)$ （高概率）。

6.3.5 桶化优化

基本构造中从 D_i 过渡到 D_{i+1} 时两者需同时存在，空间暂时翻倍。用桶化解：

- 哈希把元素分摊到 $u^{\delta/2}$ 个桶。
- 各桶数据结构按字交错存储（interleaved），保证过渡操作至多在一个桶中发生。
- 额外空间只与单个桶大小成比例。

分析单桶：底层 Raman-Rao 空间 $(1+o(1)) \cdot n' \cdot (\log(2^{\ell_i}/2^i) + r)$ 。代入 $\ell_i - i = \log(1/\varepsilon) + O(1)$ ， $r = \log \log n + O(1)$ ：

$$|D_i|_{\text{space}} = (1 + o(1))n' \log(1/\varepsilon) + O(n' \log \log n)$$

全局空间匹配下界。

6.3.6 去摊销化

不等到子序列第一个元素时一次性初始化 D_i ，而是把初始化工作均摊到该子序列的 2^i 次插入中。每次插入额外投入常数步工作。代价：不能用桶化，空间从 $(1 + o(1))n \log(1/\varepsilon) + O(n \log \log n)$ 升到 $O(n \log(1/\varepsilon) + n \log \log n)$ 。

6.3.7 支持删除（扩展）

近似成员结构无法检测是否删的是假阳性，所以要求用户保证“只删确实在集中的元素”。方案：两个字典。

- 主字典：同原结构，每个条目多一个删除指示位。
- 辅助字典：零假阳性率，存“发现是假阳性”的元素的精确值。

插入时先检查是不是假阳性，是则放辅助字典，否则正常插入主字典。删除时先查辅助字典，没有则查主字典，标记最小字典序条目为已删除。后台进程清理。

7 总结与展望

7.1 上下界紧致性

下界 $\Omega(n \log(1/\varepsilon) + n \log \log n)$ ，上界 $O(n \log(1/\varepsilon) + n \log \log n)$ ，渐近阶完全一致。 $\Omega(n \log \log n)$ 是不可消除的固有开销。

在过往方案中（如 Scalable Bloom Filter），为了保证总假阳性 $\leq \varepsilon$ ，每一级子过滤器的假阳性 ε_i 需要按 $1/i^2$ 衰减，每级都要存独立哈希函数，累加后总空间 $\Omega(n \log n)$ ，不是最优解。构造 1 同样多级但底层用近最优的 Raman-Rao 字典，空间压到 $O(n \log \log n)$ 附加项。构造 2 更进一步用单字典 + 签名扩展 + 辅助位，维持同等空间的同时做到常数时间查询。

7.2 $\log \log n$ 的意义

1. 证明了已知 n 和未知 n 这两种模型确实存在超线性空间差距。
2. $\log \log n$ 本身增长极其缓慢（ $n = 2^{64}$ 时 $\log \log n$ 仅为 6），相比旧方案的 $\log n$ 更具优势。
3. 首次给出了自适应扩容结构的固有理论存储下界，为工程实现提供了理论证明。

7.3 未来方向

- 理论：收紧 $\Omega(n \log \log n)$ 附加项的常数因子。
- 实践：构造 1 查询 $O(\log n)$ 较慢，构造 2 隐藏常数较大。期待设计同时满足最优空间和常数时间的实用数据结构，匹配本文空间下界。